

## 用户与鉴权的业务边界

gomall · 五角色视角 / 注册 / 双 token / RBAC / admin bootstrap

---

gomall · 业务侧 deck

# 今日大纲

五角色视角：鉴权到底在为谁工作

注册 + 登录 + 双 token：第一道业务边界

RBAC：30s 缓存为什么业务能接受

admin bootstrap + 三层边界路线图

双 token 失效 / 强制下线 / 多端隔离

## 五角色视角：鉴权到底在为谁工作

---

## 为什么把”用户鉴权”放在第一份 deck

- 一笔 100 元的订单，从浏览到收货要经过 20+ 个技术环节。鉴权是第一环，错了后面 19 环全是危房。
- 鉴权失败的代价不是 5xx 报错，是真金白银：
  - C 端越权下单 → 用户用 A 账号的余额买 B 账号的商品。
  - 商家越权发货 → 把别家的订单标”已发”，钱货两失。
  - admin 误授权 → 客服把全场红包发给自己。
- 所以 deck 01 不讲”怎么写 JWT 库”，讲**每条鉴权决策背后的业务取舍 + 客服话术 + SLO**。

鉴权 = 用最少的代码表达业务对”信任”的明确表态：信谁、信多久、信他能干什么。

## C 端越权下单：一个 HTTP 请求的资损

攻击者 **B** (余额 0) 用自己合法的 token 下单，却把 body 里的 user\_id 填成受害者 **A** (余额 ¥5000)：

```
1 POST /api/v1/order HTTP/1.1
2 access_token: <B's own valid token> # B's token, NOT A's
3 Content-Type: application/json
4
5 { "product_id": 88,
6   "num": 1,
7   "address_id": 999, # ship to B's home
8   "user_id": 1001 } # but owner = A's uid !
```

- 若服务端信了 body 里的 user\_id：扣 **A** 的 ¥5000、发货到 **B** 家——A 的钱、B 的货，A 凭空多一笔没下过的订单。
- 这不是一次 5xx，是追不回的资损：钱已划走、货已发出，客服面对的是“我没买过为什么扣我钱”。

gomall 只信 JWT 解出的 u.Id、无视 body 的 user\_id：order.UserID = u.Id，B 填谁都只记在 B 自己头上，攻击当场失效。

# 鉴权痛

| 角色    | 鉴权诉求                                                | 出错的真实后果                         |
|-------|-----------------------------------------------------|---------------------------------|
| C 端用户 | 一次登录管 10 天，改密码立即生效                                  | 频繁踢登 → DAU 流失                   |
| 商家    | 只看自家订单 / SKU                                        | 越权读单 → 商业泄密（订单流水就是经营机密）         |
| 运营平台  | 大促撞库不打爆 bcrypt（一个故意设计得很慢的 hash 函数，单次校验耗上百 ms 的 CPU） | 正常用户登录 502 → GMV 损失             |
| SRE   | 鉴权链路不能成性能瓶颈：鉴权是全站请求的第一道关                            | 解 token 每慢 1ms → 全站 p99 同步抬 1ms |

- 本 deck 每个 section 前都会标注**本节解决谁的痛**。
- 项目 README ”业务全景表” 各行对应的就是这几个角色，鉴权是它们的最小公约数。

# 存密码的前提：拖库是”何时”，不是”是否”

密码存储的威胁模型不是”有人在线猜密码”，而是整张 **user** 表被拖走（SQL 注入 / 内网渗透 / 备份泄露）。拖库之后比的只有一件事：把摘要还原成密码要花多少钱。

- **存明文**：拖库 = 全体用户裸奔。用户到处复用密码，你泄露的密码会被拿去撞他的支付宝。
- **MD5 / SHA-256**：这类函数为”快”而设计——一张消费级显卡每秒能算几百亿次，常见密码字典几小时扫完，等于白给。
- **加盐的快哈希**：盐只废掉彩虹表——攻击者提前把常见密码 → 摘要算成大表，拖库后直接反查；盐让每个用户都得单独重算，预计算白做。但对着单个账户现场跑字典，盐拦不住。
- **bcrypt 的答案**：把”算一次”故意变贵。合法用户登录只算 1 次（两三百 ms 无感），拖库者字典里每个词都要付两三百 ms。而且 bcrypt 每轮都要反复读写 4KB 内部状态表，GPU 的海量核心被内存访问卡死、并行加速失效——这是它敢慢的底气。

```
1 $2a$12$R9h/cIPz0gi.URNNX3kh20PST9/PgBkquzi.Ss7KIUg02t0jWMUW
2 ^^ ^^ \-----/\-----/
3 ver cost=12 22-char salt 31-char hash
```

bcrypt 三件套：**cost 旋钮** ( $2^{12}=4096$  轮，硬件变快就 +1，安全性随硬件水涨船高) · **自动加盐** (同密码不同摘要) · **自包含格式** (版本/cost/盐全编码在摘要串里——升级 cost 不用迁移存量数据，老摘要按老参数校验，用户下次登录成功时顺手 rehash 到新 cost)。

# 代码片段：bcrypt 在 gomall 的三处落点

```
1 const PassWordCost = 12 // 2^12 = 4096 轮, 单次校验数百 ms CPU
2
3 func (u *User) SetPassword(password string) error { // 注册 / 改密码
4     bytes, err := bcrypt.GenerateFromPassword([]byte(password), PassWordCost)
5     if err != nil {
6         return err // 失败绝不写脏摘要
7     }
8     u.PasswordDigest = string(bytes) // 盐已编码在摘要里
9     return nil
10 }
11 func (u *User) CheckPassword(password string) bool { // 登录 (service.go:98)
12     return bcrypt.CompareHashAndPassword(
13         []byte(u.PasswordDigest), []byte(password)) == nil
14 }
15 func (u *User) SetMoneyPassword(pin string) error { ... } // 6 位支付密码同样存摘要
```

- User 表只有 PasswordDigest 一列，没有 salt 列——CompareHashAndPassword 从摘要里解析出盐和 cost 重算比对。
- 6 位 PIN 只有 100 万种可能： $10^6 \times 300\text{ms}$ ——单核串行也只要 3.5 天，多核并行按小时计。低熵凭证 bcrypt 只是兜底，真正的防线是在线侧错误锁定 / 限流——哈希救不了熵不足。
- 彩蛋：CheckMoneyPassword 对空摘要直接放行（历史用户未设支付密码，model.go:124）——“兼容优先”的安全默认值，课堂讨论：这算 bug 还是权衡？
- 慢的反噬：撞库（拿别处泄露的账密批量试登录）不需要破解任何东西，每发一个请求就逼服务器付数百 ms CPU——几百 QPS 就能打满登录服务，正常用户 502。

同一个“慢”：离线是武器（拖库还原成本以年计），在线是软肋（CPU 被撞库打满）——所以 router.go:41 的令牌桶（每 IP 100 RPS）必须站在 bcrypt 前面，这就是“鉴权痛”表里运营平台那一行的答案。

## 业务困局：鉴权不止是“登录拿 token”

- 鉴权 = 三件事：身份 / 角色 / 边界。三件单挑都不难，难在配合：
  - 身份对了角色错 → 客服越权补券给自己。
  - 角色对了边界错 → admin 接口暴露给 C 端。
  - 身份角色都对，边界没画清 → 商家 A 看见商家 B 的订单。
- gomall 的回答：
  - 身份：双 token + bcrypt cost=12，下面 section 2 讲。
  - 角色：RBAC + 30s sync.Map 缓存，section 3 讲。
  - 边界：路由分组 + middleware 链 + admin bootstrap，section 4 讲。

## 鉴权是商业模式的边界

- 一个电商，凡是和“钱”挂钩的接口，背后都是一句话：这个请求是谁发的，他能做什么。
- gomall 把这句话拆成三件事：
  - 身份：注册 / 登录 / token 续期——客户在不在。
  - 角色：user / merchant / admin ——客户能看见什么。
  - 边界：路由分组 + middleware 链——客户做不到什么。
- 任何一件做错，损失都是真金白银：身份错则越权下单，角色错则客服把全场红包发给自己，边界错则 admin 接口暴露给 C 端。

## gomall 的三层墙：公开 / authed / admin



- 公开层：能匿名访问，挂 HTTP cache，不挂任何 auth。
- authed 层：所有 C 端用户登录后能用，挂 AuthMiddleware。
- merchant 层：当前还没有独立 **group**，商家身份借 user 表 Role 字段表达。
- admin 层：在 authed 内嵌一层 RequireRole("admin")。

# 代码片段：三层墙是怎么砌的

```
1 // routes/router.go —— 组合根：三层墙一次砌好，每层墙开一道门
2 v1 := r.Group("api/v1") // 第一道门：公开层，不设锁
3 authed := v1.Group("/")
4 authed.Use(middleware.AuthMiddleware()) // 第二道门：验 JWT（你是谁）
5 adminGroup := authed.Group("/admin")
6 adminGroup.Use(middleware.RequireRole("admin")) // 第三道门：验角色（能干嘛）
7
8 for _, register := range []func(public, authed, admin *gin.RouterGroup){
9     user.RegisterRoutes, product.RegisterRoutes, // ...共 20 个领域，统一签名自注册
10 } { register(v1, authed, adminGroup) }
```

```
1 // internal/user/routes.go —— 发凭证的门自己不能锁
2 public.POST("user/register", UserRegisterHandler())
3 public.POST("user/login", UserLoginHandler())
4 authed.GET("user/show_info", ShowUserInfoHandler())
5
6 // internal/refund/routes.go —— merchant 墙未落地的临时替身：逐路由挂 admin RBAC
7 authed.POST("orders/refund/approve",
8     middleware.RequireRole("admin"), ApproveRefundHandler())
```

- 墙只在组合根（composition root, router.go 这一个组装点）砌一次，领域包拿到的是已套好中间件的 group，只选墙、不砌墙——挂错墙在 code review 里就是一行看得见的 diff。
- public 组不是“忘了鉴权”：login / register 是发凭证的入口，必须匿名可达。
- 上一页 merchant“路线图”的现状：approve / ship 逐路由叠 RequireRole("admin") 顶着——将来 merchant 落地要一处处改回。

分组即策略：把“谁能进”编码在路由结构里（进门就查），而不是散在每个 handler 里（进屋后才想起来查）——前面「C 端越权下单」案例的第一道防线就是这里。

## 业务问题清单 + 本 deck 的回答顺序

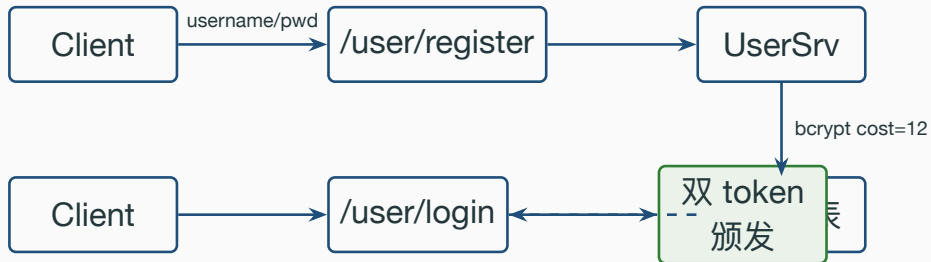
1. 注册 + 登录链路，密码加密 / 邮箱验证业务边界画在哪。
2. access 24h / refresh 10d 这两个数字怎么定，怎么影响留存。
3. RBAC 为什么敢上 30s 内存缓存。
4. admin bootstrap 这个“一次性接口”为什么必须存在。
5. C 端 / merchant / admin 三层边界，merchant 还差什么。
6. 客服补券走哪条路？为什么用户自助补券是黑产温床。
7. 业务码 30001 / 30002 / 30005 对应的客服话术。

鉴权是业务对“信任”的一次明确表态：信谁，信多久，信他能干什么。

注册 + 登录 + 双 token：第一道业务  
边界

---

## 注册 / 登录 / 颁发 token 的时序



## 密码是怎么存的: bcrypt cost=12

- **bcrypt** 自带 salt, 单向, 不可还原。重置密码只能让用户重新设置。
- PassWordCost = 12: 单次哈希 ~250ms (M 系列 Mac)。
  - 业务收益: 撞库脚本一秒只能猜 4 次, 10 位密码组合在合理时间内不可穷举。
  - 业务代价: 登录接口 RT 多 250ms, C 端可接受。
- 注册存哈希, 登录拿明文重算 hash 再 compare。后台永远拿不到明文, 被拖库后撞库代价巨大。
- 找回密码必须走邮箱验证 + token 写新摘要: EmailClaims 携带 bcrypt 摘要而非明文, TTL 15min 远短于 access。

## bcrypt cost=12 的业务后果（用钱算账）

被拖库情景：黑产拿到 password\_digest 全表，离线撞库。

配置                      1 万弱密码账号破解成本    1 万账号破解时长

明文 / MD5                      ~0 美元                      秒级

bcrypt cost=10                      ~600 美元 GPU                      ~8 小时

**bcrypt cost=12                      ~9,600 美元 GPU                      ~5 天**

bcrypt cost=14                      ~150k 美元                      ~80 天

登录 RT 代价：

cost=12 单次 ~250ms。

- 经验数据：登录页 RT 每多 100ms，注册转化率掉 ~ 0.8%，DAU 留存掉 ~ 0.3%。
- 250ms 落在“用户感知阈值 300ms”之内，转化和留存基本不动。
- 反例：cost=14 单次 1s → 用户感知“卡顿” → 大促日新用户注册流失估算 5-8%。

## 双 token 续期的状态机



- **access valid**: AuthMiddleware 顺便续发新 access + refresh, 写回 header + cookie。
- **access 过期 / refresh 有效**: 拿 refresh 重发一对新 token, 用户无感知。
- **两者都过期**: 返回 30001, C 端跳登录页。

## refresh 10d / 7d / 30d 三选一：复购漏斗 vs 安全风险

| refresh 取值 | 业务收益                          | 安全代价                           |
|------------|-------------------------------|--------------------------------|
| 7d         | 周一上班大概率重登,复购漏斗在登录页流失 ~ 8%     | 偷设备最多 7d 危险窗                   |
| 10d        | 黄金周 + 双休回流用户无感登录(电商目标用户群行为吻合) | 偷设备 10d 危险窗, 可接受               |
| 30d        | 月活用户几乎不重登,登录页流失 ~ 2%          | 偷设备 30d → 客诉激增, 黑产二手交易” 账号即资产” |

- gomall 选 10d 是业务行为画像决定的：电商用户两周内打开 app 概率 > 85%。
- 30d 看似友好，实际放大被偷账号挂闲鱼”包登录”黑产：被盗号 24h 内不修改资料，老主人完全感知不到。
- 10d 是合规线（很多支付牌照要求 token 长度  $\leq 14d$ ）。

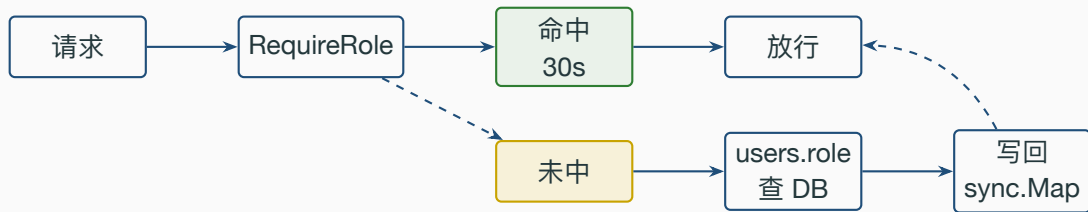
## 代码片段 1: AuthMiddleware 主链路

```
16 accessToken := c.GetHeader("access_token")
17 refreshToken := c.GetHeader("refresh_token")
18 if accessToken == "" {
19     code = e.InvalidParams
20     c.JSON(200, gin.H{"status": code, ...})
21     c.Abort(); return
22 }
23 newAccessToken, newRefreshToken, err :=
24     util.ParseRefreshToken(accessToken, refreshToken)
25 if err != nil { code = e.ErrorAuthCheckTokenFail }
26 ... // 解析成功后还有一道 token_version 撤销门 (见第五节)
27 SetToken(c, newAccessToken, newRefreshToken)
28 c.Request = c.Request.WithContext(
29     ctl.NewContext(c.Request.Context(), &ctl.UserInfo{Id: claims.ID}))
```

**RBAC: 30s 缓存为什么业务能接受**

---

## RBAC 30s 内存缓存命中链路



## 代码片段 2: lookupRole + 30s sync.Map 缓存

```
22 var (  
23     roleCache sync.Map  
24     roleCacheTTL = 30 * time.Second  
25 )  
26 func lookupRole(ctx context.Context, userId uint) (string, error) {  
27     if v, ok := roleCache.Load(userId); ok {  
28         e := v.(roleCacheEntry)  
29         if time.Now().Before(e.expires) { return e.role, nil }  
30     }  
31     u, err := user.NewUserDao(ctx).GetUserById(userId)  
32     if err != nil { return "", err }  
33     role := u.Role  
34     if role == "" { role = "user" }  
35     roleCache.Store(userId, roleCacheEntry{role, time.Now().Add(roleCacheTTL)})  
36     return role, nil  
37 }
```

## 30s 在业务上意味着什么 + 显式失效兜底

- 提权场景：管理员把客服 A 升为 admin → 最长 30s 后享受 admin 权限。
- 降权场景：admin 误授权后回收 → 误授权用户最长 30s 内仍能继续操作。
- 30s 误授权窗口能干坏事吗？
  - admin 当前接口（users / promote / backfill）不会瞬间造成资金损失。
  - 真正控钱的接口（提现 / 退款）尚未挂在 admin 子组，无 30s 资金风险。
- 必须零 TTL 的场景（封禁恶意 admin），用 InvalidateRoleCache(userId) **显式失效**。
- 选型 = **最终一致 + 关键路径显式失效**：99% 走 0ms 缓存命中，1% 走 Invalidate 立即生效。

## admin bootstrap + 三层边界路线图

---

- 上线一个全新 gomall 实例，users 表是空的。
- 想给”运维大佬”开 admin 权限，必须调 `/admin/users/promote`。
- 但 `/admin/...` 子组的中间件链是 `Auth + RequireRole("admin")`。
- 此时一个 **admin** 都没有，谁也进不去 **admin** 子组 → 没人能把别人提升为 admin。

这就是 admin bootstrap 必须存在的原因：先有第一个 **admin**，后面 **RBAC** 才能转起来。

一般在线上 db 里直接插入 sql 给予权限。

## 代码片段 3: BootstrapPromoteSelf 一次性校验

```
65 func (s *AdminSrv) BootstrapPromoteSelf(ctx context.Context) error {
66     u, err := ctl.GetUserInfo(ctx)
67     if err != nil { return err }
68     db := user.NewUserDao(ctx).DB
69     var count int64
70     if err := db.Model(&user.User{}).
71         Where("role = ?", user.RoleAdmin).
72         Count(&count).Error; err != nil {
73         return err
74     }
75     if count > 0 {
76         return errors.New("系统已存在 admin, 禁止使用 bootstrap 接口")
77     }
78     return s.PromoteToAdmin(ctx, u.Id)
79 }
```

## C 端 / merchant / admin 三层对比

| 角色       | 路由位置     | 中间件链             | 业务能力    |
|----------|----------|------------------|---------|
| 匿名 user  | v1 顶层    | 仅全局桶             | 浏览商品    |
| C 端 user | authed 组 | + AuthMiddleware | 下单 / 抢券 |
| merchant | 暂无       | —                | 路线图     |
| admin    | admin 子组 | + RequireRole    | 提权 / 后台 |

- C 端 vs admin 边界靠 `RequireRole("admin")` 拦下。
- merchant 身份当前由 user 表 Role 字段表达，单店家阶段够用；多店家时升级为独立 group。
- product/create 当前挂在 authed 组（单店家模型下由 Role 控权）；多店家阶段迁入 merchant group，叠加 `RequireRole("merchant")` + DAO 层 shop\_id 隔离。

## 攻击面：登录接口要扛三种黑产打法

| 攻击类型                     | 特征       | 单点拦截手段       |
|--------------------------|----------|--------------|
| 撞库 (credential stuffing) | 泄露库扫所有用户 | 失败限频 + 二次验证  |
| 密码喷洒 (spraying)          | 弱密码扫万人   | 全局账号失败计数     |
| 账号枚举                     | 错误码差异爆破  | 错误码归一 / 时序对齐 |

- 登录基线：bcrypt cost=12 已让单次猜测代价 ~250ms，离线撞库极不经济。
- 错误码归一：10003（用户不存在）/ 10005（密码错）对外统一话术，杜绝靠回包差异枚举用户名（对内保留具体码做审计）。
- 行业通用的下一层是**失败限频**：3000 IP 代理池 × 5 RPS = 15k RPS 的高频试探，靠 IP + 账号双维度限频在打到 bcrypt 之前就拦掉——接入点见下页。

## 撞库防御三段式：限频 + 验证码 + 设备指纹



- 第一层：IP 维度滑动窗口限频，复用 deck 10 SlidingWindow。
- 第二层：账号维度连续失败计数，5 次失败强制带验证码。
- 第三层：15min 内累计 10 次失败 → 冻结 30min，直接返回 10005 不消耗 bcrypt。
- bcrypt cost=12 每次 250ms，账号冻结后省的是 **CPU** 不是用户体验。

## 代码片段 4：登录失败计数 + 强制验证码门槛

```
1  const (  
2      loginFailKeyPrefix = "auth:login:fail:"  
3      loginFailThreshold = 5 // 15min 内  
4  )  
5  
6  func (s *UserSrv) preflight(ctx context.Context, name, cap string) error {  
7      fails, _ := cache.RedisClient.Get(ctx, loginFailKeyPrefix+name).Int()  
8      if fails >= loginFailThreshold && cap == "" {  
9          return ErrCaptchaRequired // 业务码 10011  
10     }  
11     if fails >= loginFailThreshold {  
12         if ok, _ := captchaVerify(ctx, cap); !ok { return ErrCaptchaInvalid }  
13     }  
14     return nil  
15 }
```

双 token 失效 / 强制下线 / 多端隔离

---

## token 黑名单方案：版本号 vs SET

| 方案               | 实现                             | 代价                |
|------------------|--------------------------------|-------------------|
| A. Redis SET 黑名单 | SISMEMBER jti                  | 每请求 1 RTT, TTL 难管 |
| B. 用户 token 版本号  | users.token_version, 签进 claims | 每请求查 DB / 本地缓存    |
| C. 用户最早签发时间戳     | users.token_min_iat, 对比 iat    | 改密码只写一次           |

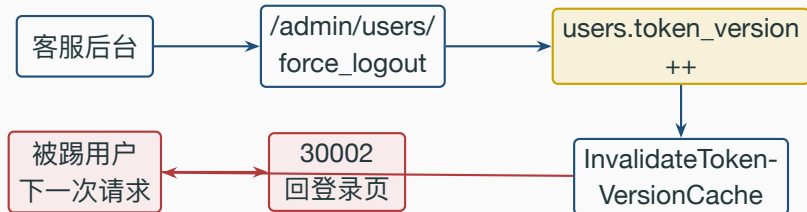
- A 按 token 粒度（被偷场景），jti 数随活跃用户线性增长。
- B/C 按用户粒度（改密码 / 强制下线），改一次字段全部作废。
- gomall 已落地 **B+C 合并**：版本号 ++ 即生效，本地 sync.Map 命中 0ms——改密码”立即生效”从路线图变成现状。

## 代码片段：B+C 落地——签发透传 / 撤销门 / 原子 bump

```
1 type Claims struct { // pkg/utils/jwt/jwt.go
2     ID uint `json:"id"`
3     Username string `json:"username"`
4     TokenVersion uint `json:"token_version"` // 与 users.token_version 比对
5     jwt.RegisteredClaims
6 }
7 // 续签透传旧版本号, 不"洗白": 被 bump 的 token 续签出的新 token 照样被拒
8 return GenerateToken(accessClaim.ID, accessClaim.Username, accessClaim.TokenVersion)
9
10 // middleware/jwt.go —— AuthMiddleware 解析成功后的撤销门 (fail-closed)
11 curVer, err := currentTokenVersion(ctx, claims.ID) // sync.Map 命中 0 DB
12 if err != nil || claims.TokenVersion != curVer { /* 拒绝, 重新登录 */ }
13
14 // internal/user/repo.go —— 版本号只允许这一条路径写入
15 func (d *UserDao) BumpTokenVersion(uId uint) error {
16     return d.DB.Model(&User{}).Where("id=?", uId).
17         UpdateColumn("token_version", gorm.Expr("token_version + ?", 1)).Error
18 }
```

- 中间件里续签发生在版本门之前——若续签时取当前版本号, 旧 token 一续签就洗白复活。续签只延长寿命、不重建信任, 重建信任只能走登录。
- 查询走组合根注入 + **sync.Map 缓存** + 主动失效——与 RBAC lookupRole 同一套三件套; 撤销门是安全边界, 故 **fail-closed** (对比 IP 限频的 fail-open)。
- 存量平滑: 旧 token 无该字段解码为 0 == 存量行默认值 0, 放行; 首次改密码才强制重登。
- 审查抓到的隐藏漏洞: UpdateUserById 整行回写会把旧版本号写回、**回滚撤销** (慢速头像上传与改密码并发即复活被盗 token) ——修复一行 `Omit("token_version")`。教训: **撤销字段必须单写路径**。

## 客服强制下线时序（接口路线图，机制已就位）



- 机制已就位：BumpTokenVersion + InvalidateTokenVersionCache 就是改密码链路在用的两个函数，接口只差一行调用 + admin 墙（路线图）。
- 失效本地缓存：与 RBAC InvalidateRoleCache 同款模式，按 userId 删 sync.Map 槽。
- 多实例场景：写 Redis pub/sub 让所有实例 Del 本地缓存槽，最终一致延迟 < 100ms。

## 为什么 gomall 选纯 JWT，不走 session cookie

| 维度      | Server Session   | JWT (双 token)       |
|---------|------------------|---------------------|
| 鉴权 RT   | 读 Redis 1 RTT    | 解 HS256 零 RTT       |
| 失效粒度    | 删 session $O(1)$ | 黑名单 / token_version |
| 水平扩缩    | 共享存储瓶颈           | 无状态，加机器即可           |
| 跨域 / 多端 | cookie SameSite  | header 跨域无障碍        |

- gomall 优先：鉴权延迟 + 水平扩展 + 多端 → JWT。
- 代价：失效慢——token\_version 版本号兜底已落地（用户粒度）；token 粒度黑名单留给”只踢一端”。
- 反例：银行类业务选 session 更合理，失效粒度比延迟重要。

## 审计日志：admin 操作必须留痕

- 当前 admin 3 个接口 (promote / backfill / users)，**无审计写入**。
- 合规字段：operator\_id / target\_id / action / reason / before / after / ts / ip / UA。
- 路线图：admin\_audit 表 + AuditMiddleware 挂 admin 子组自动入表。
- 索引：(operator\_id, ts) 客服自查；(target\_id, ts) 投诉反查。
- 不允许 DELETE，按月分区，异地冷备 7 年。审计是补券 / 强制下线的前置依赖。

## 面试 Q&A (上)

**Q1:** access 24h / refresh 10d 这两个数字怎么定?

A: access 短限泄露损失, refresh 长覆盖周末回流, 10d 是留存与安全的折中。

**Q2:** RBAC 30s 内存缓存, admin 误授权 30s 不是漏洞吗?

A: 常规延迟可接受。关键路径 InvalidateRoleCache(userId) 显式失效, 提权 / 降权立即生效。

**Q3:** 登出后老 token 还能用 24h 怎么办?

A: 改密码已根治——Claims 带 token\_version, bump 后全端立即失效。登出靠客户端丢 token; 要”登出即全端失效”复用同一个 bump。

**Q4:** bcrypt cost=12 怎么选? 怎么无痛升 14?

A: 250ms 在 300ms 感知阈值内, 穷举不现实。登录成功路径探测旧 hash 顺便 re-hash, 90 天自然升完。

**Q5:** 登录接口怎么扛 15k RPS 撞库?

A: IP + 账号双维度滑动窗口 + 5 次失败强制验证码 + 30min 冻结, bcrypt 不消耗在冻结账号上。

**Q6:** admin bootstrap 一次性接口怎么防重?

A: count(\*) WHERE role='admin', > 0 直接返回错。并发加 advisory lock 兜底。

## 面试 Q&A (下)

**Q7:** merchant 没独立 group 为什么不阻塞上线?

A: 单店家场景 user 即 merchant。多店家才需 group + DAO 层 shop\_id 双层。

**Q8:** 业务码 30001 vs 30002 用户视角一样, 区分有何意义?

A: 用户都重登, 客服区分”安全事件 vs 长闲置”, 比例异常触发审计。

**Q9:** 客服强制下线如何实现? 多端怎么隔离?

A: 机制已落地——users.token\_version 签进 claims, 改密码 ++ 全端作废; 强制下线接口复用 BumpTokenVersion (路线图)。多端按 client\_type 分桶 (路线图)。

**Q10:** 为什么 gomall 选 JWT 不选 session?

A: 零 RTT + 无状态扩展 + 多端 header 无跨域。代价是失效慢, token\_version 兜底。银行业务选 session 更合理。

**Q11:** 第三方登录怎么接入?

A: OAuth 拿 external\_id → 查/建 user\_oauth → 复用 GenerateToken。后续仍走 AuthMiddleware。

**Q12:** 100k QPS 鉴权怎么扩? 瓶颈在哪?

A: HS256 + sync.Map 单机 58k RPS, 100k 扩 4 实例够用, 瓶颈始终在下单 / 支付 / 库存。

## 课后作业

---

## 课后作业·编程题（可上 OJ 判题）

### 题 1 ·越权下单的身份判定 (IDOR)

下单请求同时带两处身份：JWT 解出的可信 uid，与请求体里的 body\_uid。实现 resolveOwner，返回订单真正的归属用户，并说明为何忽略 body\_uid。

输入：每行 uid body\_uid（多组） 输出：每组订单归属 uid

判分点：无论 body\_uid 填谁，归属恒等于 uid（信 JWT，不信报文）。

### 题 2 ·双 token 续期状态机

给定 access\_ttl / refresh\_ttl 与一串带时间戳的请求，输出每次请求后的动作 PASS / REFRESH / RELOGIN：  
access 过期且 refresh 有效 → REFRESH；两者皆过期 → RELOGIN。

### 题 3（选做）·登录失败计数 + 强制验证码

滑动窗口内失败达阈值即要求验证码，冻结期请求直接拒绝、不消耗 bcrypt。输入事件流，逐条输出：放行 / 要求验证码 / 冻结。

## 课后作业·概念思考题（检测是否学懂）

用一两句话回答，重点讲“为什么”，不是“是什么”：

1. bcrypt 为什么选 cost=12，而不是更快的 10 或更慢的 14？升到 14 如何做到用户无感？
2. 登出后老 access 还能用 24h，为什么这不算致命漏洞？兜底手段是什么？
3. RBAC 用 30s 内存缓存，admin 误授权 30s 才生效——为什么业务能接受？哪条路径必须绕过缓存立即失效？
4. admin 冷启动悖论是什么？BootstrapPromoteSelf 用什么条件保证只能被“占用”一次？
5. 下单接口为什么只信 JWT 里的 uid、无视请求体的 user\_id？若信了会发生什么资损（用 A 的余额买 B 的货）？
6. gomall 为什么用无状态 JWT 而不用 session？这个选择在什么业务（如银行）下应该反过来？